

Exercise 8

Feedforward Neural Networks

In [27]:

```
import numpy as np

#####
#
#           Functions to generate data
#
#
#####

def func(X: np.ndarray) -> np.ndarray:
    """The data generating function"""
    return 0.3 * X[:, 0] + 0.6 * X[:, 1] ** 2

def noisy_func(X: np.ndarray, epsilon: float = 0.075) -> np.ndarray:
    """Add noise to the data generating function"""
    return func(X) + np.random.randn(len(X)) * epsilon

def get_data(n_train: int, n_test: int) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:
    """Provide training and test data for training the neural network"""
    X_train = np.random.rand(n_train, 2) * 2 - 1
    y_train = func(X_train)
    X_test = np.random.rand(n_test, 2) * 2 - 1
    y_test = func(X_test)

    y_train = y_train.reshape((y_train.shape[0],1))
    y_test = y_test.reshape((y_test.shape[0],1))

    return X_train, y_train, X_test, y_test

#####
#
#           Feed-forward Neural Network
#
#
#####

def sigmoid(x):
    """
    Sigmoid activation function
    """
    # TODO implement sigmoid activation function
    sig = 1/(1+np.exp(-x))
    return sig

def d_sigmoid(y):
    """
    Derivative of the sigmoid activation function - input y is output of
```

```

"""
# TODO implement derivative of sigmoid activation function
dsig = y*(1-y)
return dsig

def linear(x):
    """
    linear activation function
    """
    return x

def d_linear(y):
    """
    Derivative of linear activation function
    """
    return np.ones_like(y)

def mse(y, y_hat):
    """
    Mean squared error loss function
    """
    return np.mean((y - y_hat) ** 2)

def feed_forward(x, hidden_W, hidden_b, out_W, out_b):
    """
    Calculate the forward pass for input_data x
    """

    # Hidden layer calculations
    # TODO implement the forward pass through the hidden layer
    lin_trans = np.dot(x, hidden_W) + hidden_b
    hidden_activations = np.maximum(0, lin_trans) # ReLU Activation

    # Output calculations
    out_linear = np.dot(hidden_activations, out_W) + out_b
    y_hat = sigmoid(out_linear)

    return y_hat, hidden_activations

def backward(y_hat, y, hidden_activations, x, out_W):
    """
    Backpropagation for the neural net in the assignment
    Parameters are those needed for the calculation

    :param y_hat: predictions from forward pass
    :param y: target data
    :param hidden_activations: Activations of hidden layer from forward
    :param x: input data
    :param out_W: current output weights

    :returns: the derivative updates to hidden_W, hidden_b, out_W and ou
    """

    # TODO implement backpropagation

```

```

error = y_hat - y
loss = mse(y=y, y_hat=y_hat)

dy_hat = 2*error/y.shape[0]
d_out_lin = dy_hat * d_sigmoid(y=y_hat)

# gradients wrt output weights
d_L_d_ow = np.dot(hidden_activations.T, d_out_lin) # Size (2,1)
# gradients wrt output bias
d_L_d_ob = np.sum(d_out_lin, axis=0, keepdims=True) # Size (1,)
# gradients wrt hidden weights
d_hidden = np.dot(d_out_lin, out_W.T) * (hidden_activations > 0) #
d_L_d_hw = x.T @ d_hidden # Size (2,2)
# gradients wrt hidden bias
d_L_d_hb = np.sum(d_hidden, axis=0, keepdims=True) # Size (2,)

return d_L_d_hw, d_L_d_hb, d_L_d_ow, d_L_d_ob

def train(x_train, y_train, neural_net, learning_rate=0.01, epochs=10, batch_size=10):
    """
    Train neural network on data
    """

    hidden_W, hidden_b, out_W, out_b = neural_net

    n_batches = int(np.ceil(x_train.shape[0]/float(batch_size)))

    for e in range(epochs):

        errors = []
        learning_rate *= 0.99
        for i in range(n_batches):

            # Forward pass
            y_hat, hidden_activations = feed_forward(x_train[batch_size*i:batch_size*(i+1)],
                                                       hidden_W, hidden_b, out_W, out_b)

            # Compute error
            error = mse(y_train[batch_size*i:batch_size*(i+1)], y_hat)
            errors.append(error)

            # Backward pass
            d_L_d_hw, d_L_d_hb, d_L_d_ow, d_L_d_ob = backward(y_hat=y_hat, y_train=y_train,
                                                                hidden_act=hidden_activations,
                                                                x=x_train[batch_size*i:batch_size*(i+1)],
                                                                out_W=out_W, out_b=out_b)

            # Update parameters
            # TODO update parameters using gradients returned above and
            hidden_W -= learning_rate * d_L_d_hw
            hidden_b -= learning_rate * d_L_d_hb
            out_W -= learning_rate * d_L_d_ow
            out_b -= learning_rate * d_L_d_ob

        print(f"Epoch {e+1}: mse {np.mean(errors):4f}", end="\n")

    return np.mean(errors), (hidden_W, hidden_b, out_W, out_b)

```

In [31]:

```

if __name__ == "__main__":

    np.random.seed(0)
    X_train, y_train, X_test, y_test = get_data(n_train=280, n_test=120)

    # Initialize weights for two hidden neurons
    # 2 X 2 W + 2 b
    hidden_weights = np.random.uniform(-0.5, 0.5, (2, 2))
    hidden_bias = np.random.uniform(-0.5, 0.5, (1, 2))

    # Initialize weights for output neuron
    # 2 x 1 W + 1 b
    out_weights = np.random.uniform(-0.5, 0.5, (2, 1))
    out_bias = np.random.uniform(-0.5, 0.5, (1, 1))

    # This is the neural net
    neural_net = (hidden_weights, hidden_bias, out_weights, out_bias)
    # Training
    train_mse, neural_net_trained = train(X_train, y_train, neural_net,
                                          batch_size=10)

    # Calculate mse on test data
    y_hat_test, _ = feed_forward(X_test, *neural_net_trained)
    test_mse = mse(y_test, y_hat_test)

    print(f"Training MSE: {train_mse:.6f}")
    print(f"Test MSE: {test_mse:.6f}")

```

```

Epoch 1: mse 0.116664
Epoch 2: mse 0.094244
Epoch 3: mse 0.083218
Epoch 4: mse 0.077289
Epoch 5: mse 0.073850
Epoch 6: mse 0.071709
Epoch 7: mse 0.070283
Epoch 8: mse 0.069273
Epoch 9: mse 0.068515
Epoch 10: mse 0.067914
Epoch 11: mse 0.067413
Epoch 12: mse 0.066976
Epoch 13: mse 0.066577
Epoch 14: mse 0.066201
Epoch 15: mse 0.065833
Epoch 16: mse 0.065465
Epoch 17: mse 0.065089
Epoch 18: mse 0.064697
Epoch 19: mse 0.064286
Epoch 20: mse 0.063849
Epoch 21: mse 0.063383
Epoch 22: mse 0.062884
Epoch 23: mse 0.062360
Epoch 24: mse 0.061831
Epoch 25: mse 0.061293
Epoch 26: mse 0.060739
Epoch 27: mse 0.060163
Epoch 28: mse 0.059569
Epoch 29: mse 0.058957
Epoch 30: mse 0.058327
Epoch 31: mse 0.057692
Epoch 32: mse 0.057070

```

Epoch 399: mse 0.033666
Epoch 400: mse 0.033664
Training MSE: 0.033664
Test MSE: 0.036103